ASE 330M HW1 01/26/26

1 a) $f(x) = x^2$

i. $f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)(x-a)^2}{2} \ldots$

$= \lim_{n\to\infty} \sum_{i=0}^{n} \frac{f^{i}(a)(x-a)^{i}}{i!}$

about $x=0 \Rightarrow a=0$.

$f(x) = f(0) + f'(0)x + \frac{f''(0)x^2}{2} + \frac{f^3(0)x^3}{3!} \ldots$

$f(x) = x^2 \rightarrow f(0) = 0$

$f'(x) = 2x \rightarrow f'(0) = 0$

$f''(x) = 2 \rightarrow f''(0) = 2$

$f^3(x) = 0 \rightarrow f^3(0) = 0$

$$\boxed{f(x) = \frac{0x^0}{0!} + \frac{0}{1!}x^1 + \frac{2}{2!}x^2 + \frac{0x^3}{3!} + \ldots \frac{0x^n}{n!} = x^2}$$

after 3 terms, the Taylor approximation is exact.

ii about $x=1 \Rightarrow (a=1)$

$f(x) = f(1) + f'(1)\cdot(x-1) + \frac{f''(1)(x-1)^2}{2} + \frac{f^3(1)(x-1)^3}{3!}$.

$f(1) = 1$

$f'(1) = 2$

$f''(1) = 2$

$f^3(x) = 0$

$f^n(x) = 0$ for $n \geq 3$.

$$\boxed{\begin{aligned} f(x) &= 1 + 2(x-1) + \frac{2(x-1)^2}{2} + \frac{0\cdot(x-1)^3}{3!} + \frac{0(x-1)^4}{4!} \ldots \frac{0(x-1)^n}{n!} \\ &\Downarrow \\ f(x) &= 1 + (2x-2) + (x^2-2x+1) = x^2 \end{aligned}}$$

Still exact after 3 terms of Taylor Series

iii. $f(x)=x^2$ is a 2nd order polynomial so all derivatives of order ≥ 3 are 0. So, after the quadratic term, the Taylor series will stop, leading to a finite series; the different centering points result in different basis polynomials x^k vs. $(x-1)^k$ but both series represent the same function and simplify to x^2 exactly → Taylor's theorem guarentees that the Taylor polynomial will exactly reproduce the function regardless of expansion point, if sufficient derivatives are considered.

1b)

i $f(x)=e^{ax}$ about $x=0$.

$f(0)=e^0=1$

$f'(x)=e^{ax}\cdot a$; $f^n(x)=a^n e^{ax}=a^n f(x)$

$f'(0)=e^0 a=a$; $f^n(0)=a^n$

$$f(x)=f(0)+f'(0)(x-0)+\frac{f''(0)(x-0)^2}{2!}\ldots+\frac{f^n(0)(x-0)^n}{n!}$$

$$\boxed{f(x)=1+ax+\frac{a^2x^2}{2!}+\ldots\frac{a^nx^n}{n!}}$$

$$\boxed{f(x)=\lim_{n\to\infty}\sum_{i=0}^{n}\frac{a^nx^n}{n!}}$$

ii $f(x)=\sin(ax)$ about $x=0$ $\quad f(0)=0$

$f'(x)=a\cos(ax)$ $\quad f'(0)=a$

$f''(x)=-a^2\sin(ax)$ $\quad f''(0)=0$

$f^3(x)=-a^3\cos(ax)$ $\quad f^3(0)=-a^3$

$f^4(x)=a^4\sin(ax)$ $\quad f^4(0)=0$

$$\sin(ax) = \lim_{n\to\infty} \sum_{i=0}^{n} \frac{a^{2n+1}\, x^{2n+1}}{(2n+1)!} (-1)^n = \lim_{n\to\infty} \sum_{i=0}^{n} \frac{(ax)^{2n+1} (-1)^n}{(2n+1)!}$$

iii

$f(x) = \cos(ax)$ $\quad f(0) = 1$

$f'(x) = -a\sin(ax)$ $\quad f'(0) = 0$

$f''(x) = -a^2\cos(ax)$ $\quad f''(0) = -a^2$

$f^3(x) = a^3\sin(ax)$ $\quad f^3(0) = 0$

$f^4(x) = a^4\cos(ax)$ $\quad f^4(0) = a^4$

$$\boxed{\cos(ax) = \lim_{n\to\infty} \sum_{i=0}^{n} \frac{(ax)^{2n} (-1)^n}{2n!}}$$

2 A linear function is a map f from $\mathbb{R}^n \to \mathbb{R}^m$ where

- $f(c_1 x_1 + c_2 x_2) = c_1 f(x_1) + c_2 f(x_2)$ $\quad \forall\, c_1, c_2 \in \mathbb{R}$ &
- $f(0) = 0$. (function passes through origin) $\quad \forall\, x_1, x_2 \in \mathbb{R}^n$

a $f(x_1, x_2) = 3x_1 - x_2 + 1$ $\quad$ let $c_1, c_2 = 1, 1$

$f(a, b) = 3a - b + 1$

where $x_1, x_2 \in \mathcal{X}_1, \mathcal{X}_2$ & $a, b \in \mathcal{X}_1, \mathcal{X}_2$.

$$f(a+x_1, b+x_2) = 3(x_1 + a) - (x_2 + b) + 1$$
$$= 3x_1 - x_2 + 1 + 3a - b$$
$$= f(x_1, x_2) + f(a, b) - 1 \neq f(x_1, x_2) + f(a, b)$$

So, $f(x_1, x_2) = 3x_1 - x_2 + 1$ is a non linear function

alternatively, $f(0,0) = 1$ does not pass through origin.

b $f(x_1, x_2) = x_1 x_2$

$a_1 \in X_1$ & $a_2 \in X_2$, where $x_1 \in X_1$ & $x_2 \in X_2$

$$f(x_1 + a_1, x_2 + a_2) = (x_1 + a_1)(x_2 + a_2) = x_1 x_2 + a_1 a_2 + x_1 a_1 + x_2 a_2$$

$$= f(x_1, x_2) + f(a_1, a_2) + x_1 a_1 + x_2 a_2$$

$$\neq f(x_1, x_2) + f(a_1, a_2)$$

So, $f(x_1, x_2) = x_1 x_2$ is non linear

c $f(g) = \frac{dg(x)}{dx} \rightarrow f(0) = \frac{d0}{dx} = 0$ also:

$$f(g+h) = \frac{d(g(x) + h(x))}{dx} = \frac{dg(x)}{dx} + \frac{dh(x)}{dx}$$

$$= f(g) + f(h)$$

So, $f(g) = \frac{dg(x)}{dx}$ is linear.

d $f(g) = \int_\alpha^\beta g(x)\,dx \rightarrow f(0) = \int_\alpha^\beta 0\,dx = 0$ also:

$$f(ag + bh) = \int_\alpha^\beta ag(x) + bh(x)\,dx \quad , a, b \in \mathbb{R}$$

$a, b \in \mathbb{R}$
$g \in G$ &
$h \in G$

$$= \int_\alpha^\beta ag(x)\,dx + \int_\alpha^\beta bh(x)\,dx$$

$$= a\int_\alpha^\beta g(x)\,dx + b\int_\alpha^\beta h(x)\,dx$$

$$= af(g) + bf(h)$$

So, $f(g) = \int_\alpha^\beta g(x)\,dx$ is linear.

3. a system $\dot{x} = f(x, u)$
is linear if it can be written in the form $\dot{x} = Ax$ for constant matrices A, B or operators
where x is the state.

$\dot{P}_x = v_x$, $\dot{P}_y = v_y$, $\dot{P}_z = v_z$ are linear

but,
1) $\dot{v}_x = g\tan\theta$, 2) $\dot{v}_y = -g\tan(\phi)$, 3) $\dot{v}_z = \tau - g$

are non linear systems since:

g is a scalar constant, so f is not parameterized by g

1) $f(\theta) = g\tan\theta$ is a non-linear function

$$f(\theta_1+\theta_2) = g\tan(\theta_1+\theta_2) \neq g\tan\theta_1 + g\tan\theta_2$$
$$= g(\tan\theta_1 + \tan\theta_2)$$
$$= f(\theta_1) + f(\theta_2)$$

for example, let $\theta_1 = 30^\circ$ & $\theta_2 = 30^\circ$

$\tan(\theta_1+\theta_2) = \tan(60^\circ) = \frac{\sqrt{3}\ 2}{2 \cdot 1} = \sqrt{3}$

$\tan(\theta_1) + \tan(\theta_2) = 2 \cdot \tan(30^\circ)$

$= 2\ \frac{1/2}{\sqrt{3}} \cdot 2 = \frac{1}{\sqrt{3}}$

2) $f(\phi) = -g\tan\phi$ is also non-linear by the same argument in 1). $f(\phi_1+\phi_2) = -g\tan(\phi_1+\phi_2)$
$\neq g[\tan\phi_1 + \tan\phi_2]$
$= f(\phi_1) + f(\phi_2)$

3) $\dot{v}_z = \tau - g$

$f(\tau) = \tau - g$; $f(\tau=0) = 0 - g = -g \neq 0$.

so, $f(\tau)$ is non-linear.

Variables θ, ϕ, and τ make the system non-linear.

With a small angle approximation,

$\tan(\theta) \approx \theta$, $\tan(\phi) \approx \phi$

so, we can get a linear affine model (still not linear!)

(only the $\dot{v}_z = \tau - g$ non-linearity)

b)

$$\frac{d}{dt}\begin{bmatrix} p_x \\ p_y \\ p_z \\ v_x \\ v_y \\ v_z \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ v_z \\ g\tan\theta \\ -g\tan\phi \\ \tau - g \end{bmatrix}$$

define $\tilde{\theta} = \tan\theta$, $\tilde{\phi} = \tan\phi$, & $\tilde{\tau} = \tau - g$

which is valid when $\theta, \phi \in (-\pi/2, \pi/2)$.

then:

$$\frac{d}{dt}\begin{bmatrix} p_x \\ p_y \\ p_z \\ v_x \\ v_y \\ v_z \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ v_z \\ g\tilde{\theta} \\ -g\tilde{\phi} \\ \tilde{\tau} \end{bmatrix}$$

let $x = \begin{bmatrix} p_x \\ p_y \\ p_z \\ v_x \\ v_y \\ v_z \end{bmatrix}$ then we can write the system of equations in $\dot{x} = Ax + B\underset{\sim}{u}$

inputs to system

$u = \begin{bmatrix} \tilde{\theta} \\ \tilde{\phi} \\ \tilde{\tau} \end{bmatrix}$

$$\dot{x} = \frac{d}{dt}\underbrace{\begin{bmatrix} p_x \\ p_y \\ p_z \\ v_x \\ v_y \\ v_z \end{bmatrix}}_{x} = \overbrace{\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}}^{A} \overbrace{\begin{bmatrix} p_x \\ p_y \\ p_z \\ v_x \\ v_y \\ v_z \end{bmatrix}}^{x} + \underbrace{\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ g & 0 & 0 \\ 0 & -g & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{B} \underbrace{\begin{bmatrix} \tilde{\theta} \\ \tilde{\phi} \\ \tilde{\tau} \end{bmatrix}}_{u}$$

The coordinate change eliminates the non-linear functions (tan) and replaces them with additional inputs $\vec{u}$.

4. $\ddot{\theta}_1 = -2\theta_1 + \theta_2$

$\ddot{\theta}_2 = 2\theta_1 - 2\theta_2$

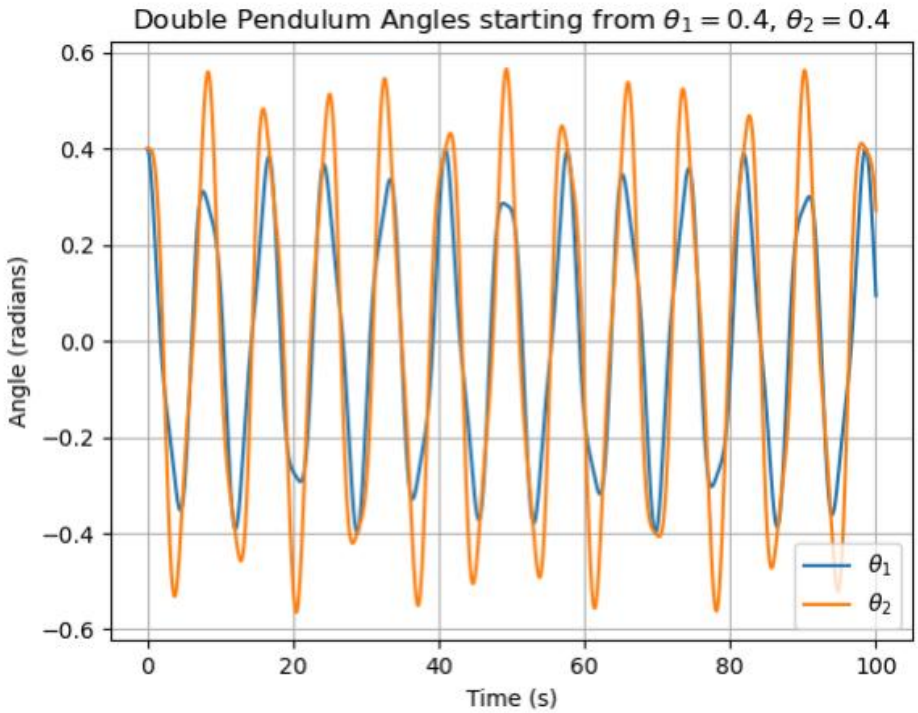
$$\frac{d}{dt}\begin{bmatrix} \theta_1 \\ \theta_2 \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -2 & 1 & 0 & 0 \\ 2 & -2 & 0 & 0 \end{bmatrix} \begin{bmatrix} \theta_1 \\ \theta_2 \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}$$

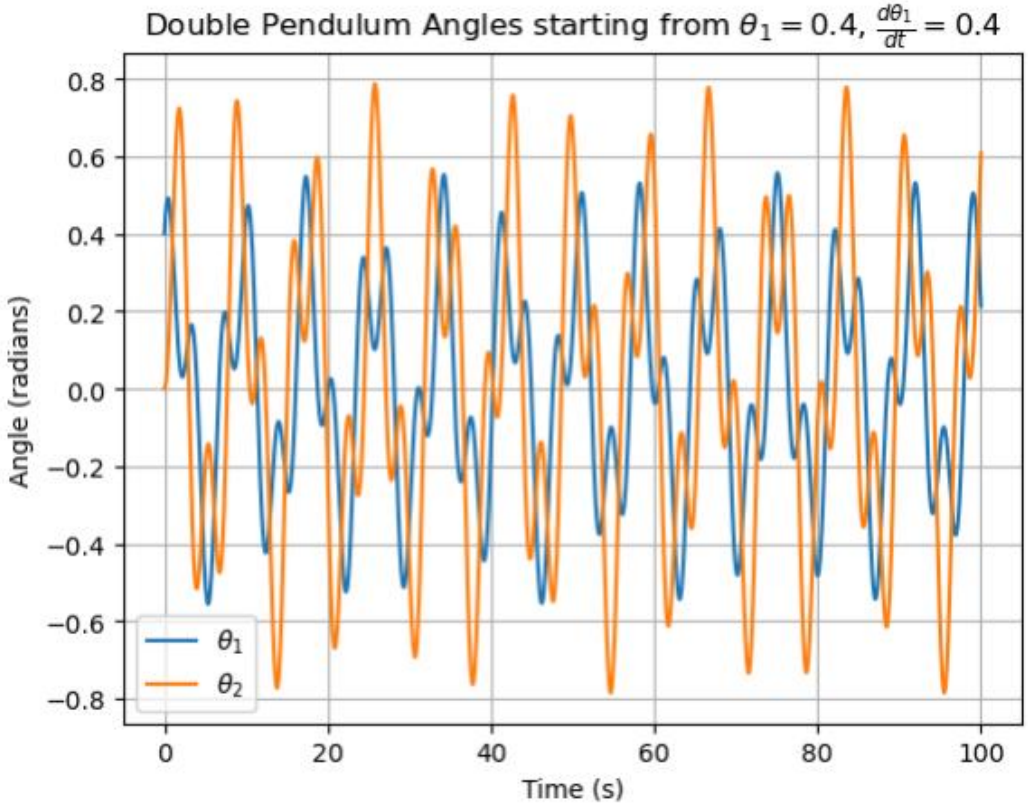
a) state variables: $\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2$

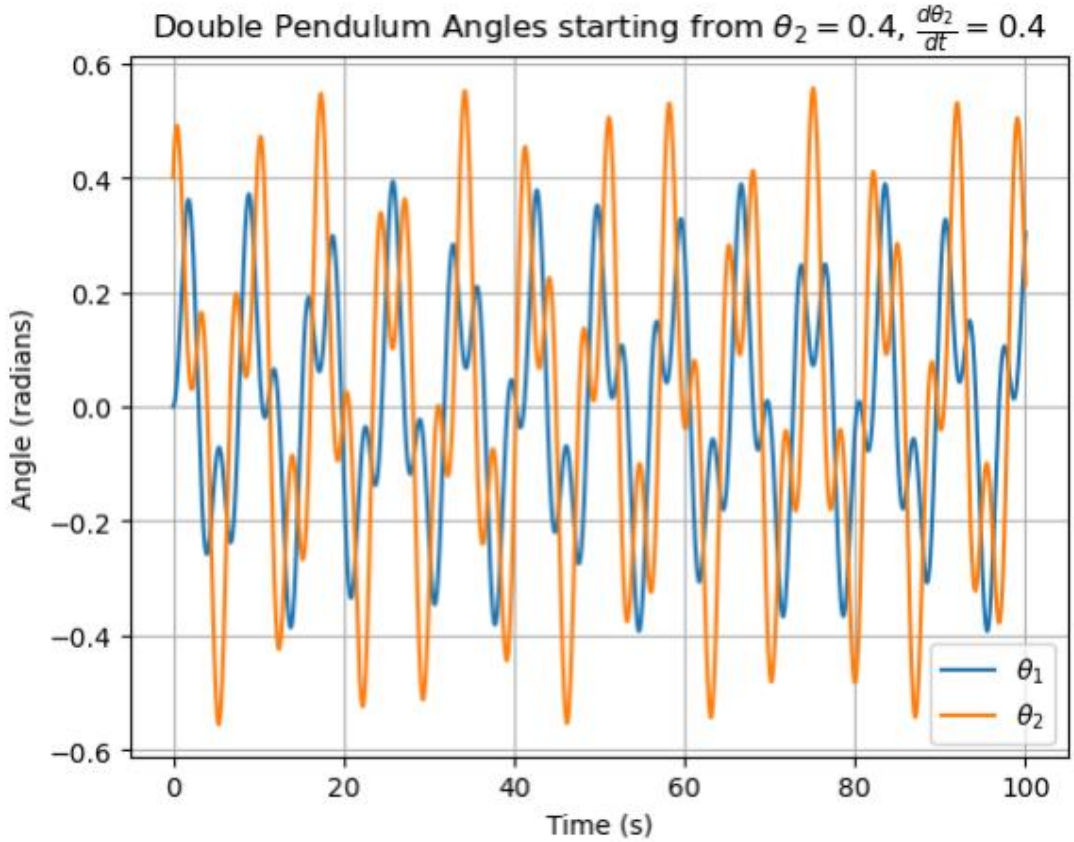
4b)

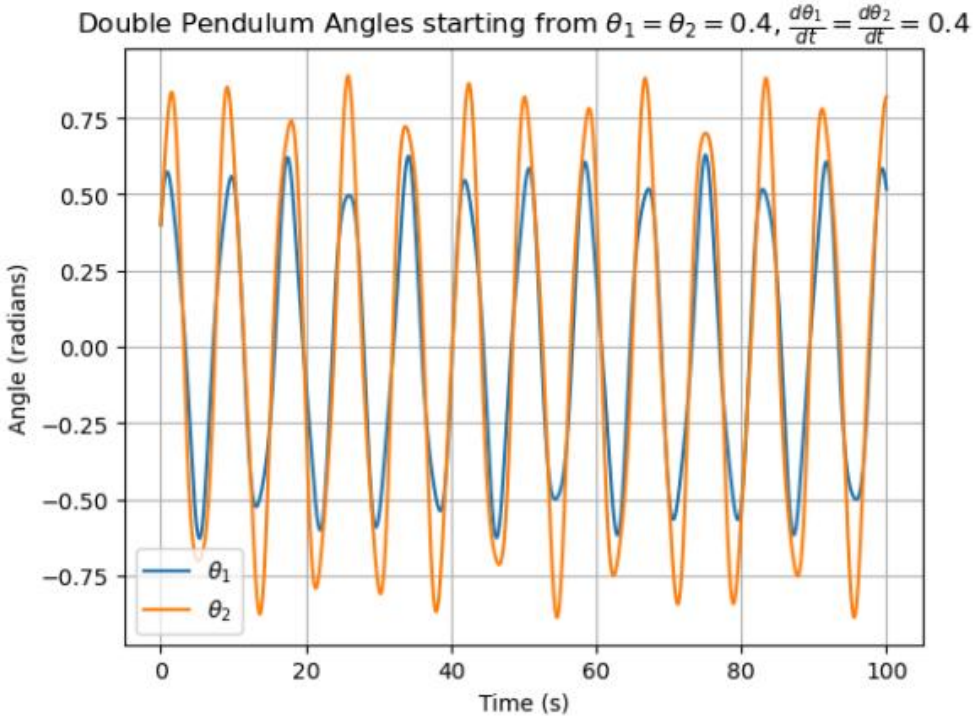
$$\frac{d}{dt}\underbrace{\begin{bmatrix}\theta_1\\ \theta_2\\ \dot\theta_1\\ \dot\theta_2\end{bmatrix}}_{x} = \underbrace{\begin{bmatrix}0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\\ -2 & 1 & 0 & 0\\ 2 & -2 & 0 & 0\end{bmatrix}}_{A}\underbrace{\begin{bmatrix}\theta_1\\ \theta_2\\ \dot\theta_1\\ \dot\theta_2\end{bmatrix}}_{x}$$

$$\frac{d}{dt}x = A\,x$$




Double Pendulum Angles starting from $\theta_1 = 0.4$, $\frac{d\theta_1}{dt} = 0.4$
Angle (radians)
Time (s)
θ_1
θ_2


Double Pendulum Angles starting from $\theta_2 = 0.4$, $\frac{d\theta_2}{dt} = 0.4$
Angle (radians)
Time (s)
θ_1
θ_2



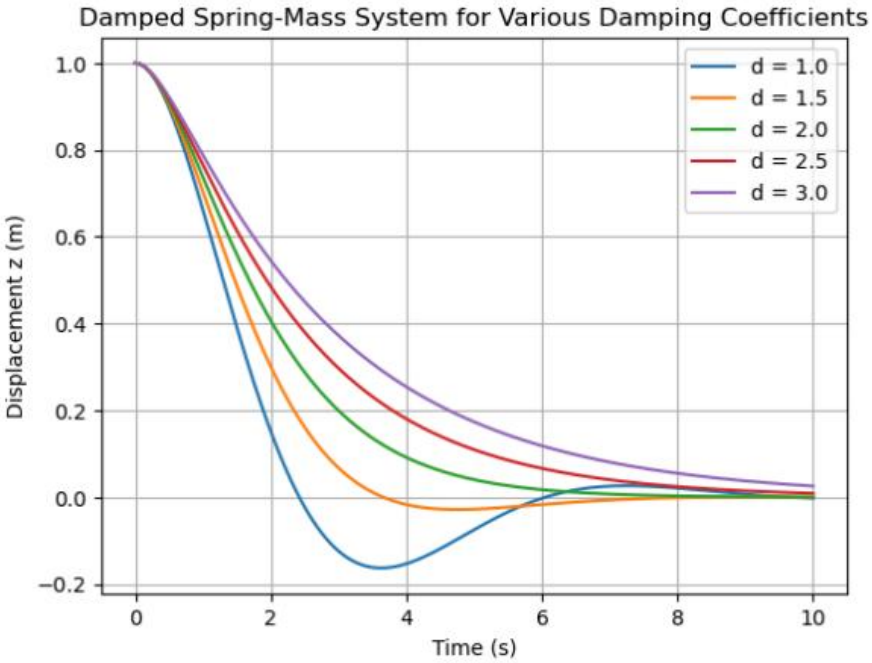
For the code, please refer to the appendix.

5. $m\ddot{z} + d\dot{z} + kz = 0 \rightarrow \ddot{z} = -\frac{d}{m}\dot{z} - \frac{k}{m}z$

a) State variables : $z, \dot{z}$

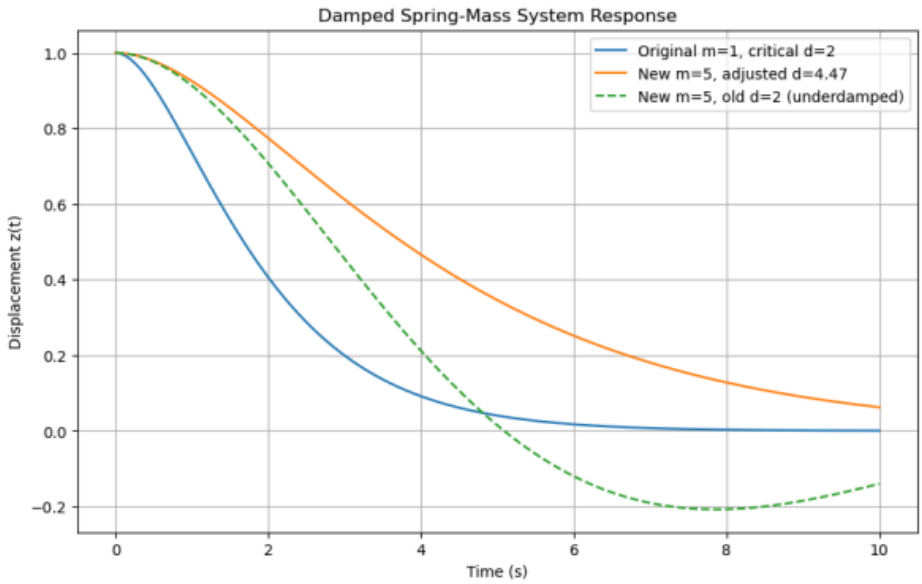
b) $\dot{x} = Ax$ format:

$$\frac{d}{dt}\underbrace{\begin{bmatrix} z \\ \dot{z} \end{bmatrix}}_{\vec{x}} = \begin{bmatrix} \dot{z} \\ \ddot{z} \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{d}{m} \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} z \\ \dot{z} \end{bmatrix}}_{\vec{x}}$$

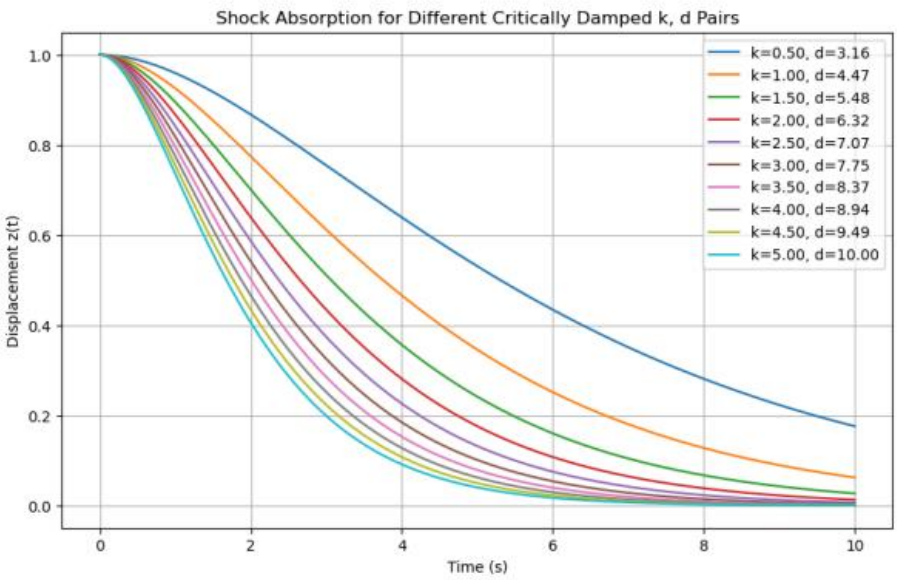


At $d = 2\sqrt{mk}$, $d = 2$ since $m = k = 1$. So, we are considering the green line. At critical damping, the system returns to equilibrium (displacement $z = 0$) as quickly as possible without any overshooting; as seen by the yellow line ($d = 1.5$), when the system is underdamped (damping less than the critical damping), there is quick return towards equilibrium, but the system overshoots and oscilates around equilibrium before settling. An overdamped system ($d > 2$, red and purple lines) will return to the equilibrium without oscillation but has a slower return to equilibrium than the critically damped system.

To understand why critical damping is ideal, consider a car's shock absorber. If the system is underdamped, the car bounces up and down (oscillates vertically) after hitting a bump. If the system is overdamped, the car slowly returns to its normal/equilibrium position, and may feel "heavy","sticky", or "sluggish." Critical damping provides the optimal balance, where the car quickly stabilizes without bouncing. Critical damping provides the fastest non-oscillatory return to a system's equilibrium.



$d_{crit} = 2\sqrt{mk}$. For critical damping, $d_{new} = 2\sqrt{5k_{new}}$.



As k increases, d also increases to maintain critical damping. Smaller spring constants k with smaller damping coefficients d lead to slower responses while larger k, d pairs lead to stiffer springs/heavier damping and faster responses. All critically damped systems have no oscilation. The "best" system depends on the application: for comfort, choose a small k and small d, but for fast response, choose large k, d pairs.

For code, please refer to appendix.

6.

a) $\ddot{z} = -z$; state variable: $z, \dot{z}$

$$\frac{d}{dt}\begin{bmatrix} z \\ \dot{z} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}\begin{bmatrix} z \\ \dot{z} \end{bmatrix}$$

$$\frac{d}{dt}\underbrace{\vec{z}}_{} = \underbrace{A}_{} \underbrace{\vec{z}}_{}$$

Alternate: any linear invertible transform of $(z, \dot{z})$ also works. e.g.:

$$\frac{d}{dt}\begin{bmatrix} z+\dot{z} \\ \dot{z} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} z \\ \dot{z} \end{bmatrix}$$

b) $\ddot{z} - \dot{z} + 2z = 0$ $\qquad$ $\ddot{z} = \dot{z} - 2z$

$$\frac{d}{dt}\underbrace{\begin{bmatrix} z \\ \dot{z} \end{bmatrix}}_{\vec{z}} = \underbrace{\begin{bmatrix} 0 & 1 \\ -2 & 1 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} z \\ \dot{z} \end{bmatrix}}_{\vec{z}}$$

$$\frac{d}{dt}\vec{z} = A\vec{z}$$

state variables $z, \dot{z}$

state variables: $z, \dot{z}$

(again, any linear invertible transform like $\frac{d}{dt}\begin{bmatrix} az \\ \dot{z} \end{bmatrix} = \begin{bmatrix} 0 & a \\ -\frac{2}{a} & 1 \end{bmatrix}\begin{bmatrix} az \\ \dot{z} \end{bmatrix}$ also works)

c) $\ddot{z} + z = e^{2t}$

$$\frac{d}{dt}\underbrace{\begin{bmatrix} z \\ \dot{z} \\ e^{2t} \end{bmatrix}}_{\vec{z}} = \underbrace{\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 1 \\ 0 & 0 & 2 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} z \\ \dot{z} \\ e^{2t} \end{bmatrix}}_{\vec{z}}$$

$$\frac{d}{dt}\vec{z} = A\vec{z}$$

state variables $z, \dot{z}, e^{2t}$

other option: treat e^{2t} as a linear-time-varying input: (affine) (not-self-contained).

$$\frac{d}{dt}\begin{bmatrix} z \\ \dot{z} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}\begin{bmatrix} z \\ \dot{z} \end{bmatrix} + \begin{bmatrix} 0 \\ \underbrace{u(t)}_{e^{2t}} \end{bmatrix}$$

d) $\ddot{z} = \cos(t)$

$$\frac{d}{dt}\begin{bmatrix} z \\ \dot{z} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}\begin{bmatrix} z \\ \dot{z} \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix}\cos t$$

Time dependent system

for linearity, instead, use:

$$\frac{d}{dt}\underbrace{\begin{bmatrix} z \\ \dot{z} \\ \cos t \\ -\sin t \end{bmatrix}}_{\vec{z}} = \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \end{bmatrix}}_{A}\underbrace{\begin{bmatrix} z \\ \dot{z} \\ \cos t \\ -\sin t \end{bmatrix}}_{z}$$

state variables: $z, \dot{z}, \cos t, \sin t$

$$\frac{d}{dt}\vec{z} = A\, z$$

With <u>fewer variables</u>, the linear system can be written as: (state space: $\dot{z}$, $\cos t$, $\sin t$).

$$\frac{d}{dt}\underbrace{\begin{bmatrix} \dot{z} \\ \cos t \\ \sin t \end{bmatrix}}_{\vec{z}} = \underbrace{\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}}_{A}\underbrace{\begin{bmatrix} \dot{z} \\ \cos t \\ \sin t \end{bmatrix}}_{\vec{z}}$$

$$\frac{d}{dt}\vec{z} = A\,\vec{z}$$

another option would be to multiply the state space by any constant $c \in \mathbb{R}$.

hw1

Exported 1/25/2026, 1:23:05 PM

Appendix - Code and Plots for Questions 4 and 5

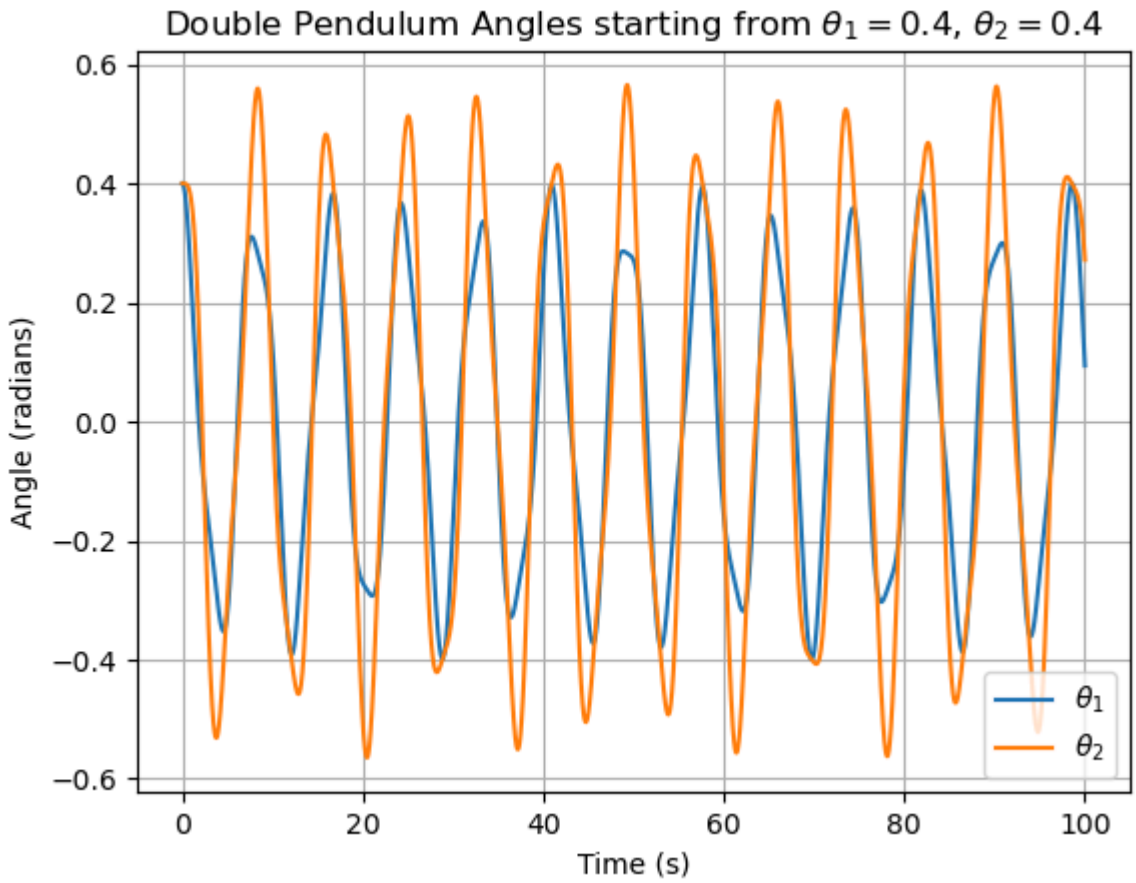
4 - double pendulum

The document was converted with **Code-Format**: https://code-format.com/

In [1]:

```
import numpy as np
from scipy.integrate import odeint
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

# State-space matrix for linearized double pendulum
A = np.array([[0, 0, 1, 0],
              [0, 0, 0, 1],
              [-2, 1, 0, 0],
              [2, -2, 0, 0]])

# Define the ODE function
def f(x, t):
    return A @ x

# Initial condition: small angles and zero velocities
x0 = np.array([0.4, 0.4, 0, 0])

# Time vector
t = np.linspace(0, 100, 5000)

# Solve the ODE
x = odeint(f, x0, t)

# Extract angles
theta1 = x[:, 0]
theta2 = x[:, 1]

# Plot angles
plt.figure()
plt.plot(t, theta1, label=r'$\theta_1$')
plt.plot(t, theta2, label=r'$\theta_2$')
plt.xlabel('Time (s)')
plt.ylabel('Angle (radians)')
plt.legend()
plt.grid(True)
plt.title(r'Double Pendulum Angles starting from $\theta_1 = 0.4$,
$\theta_2 =0.4$')
plt.show()
```

The document was converted with **Code-Format**: https://code-format.com/

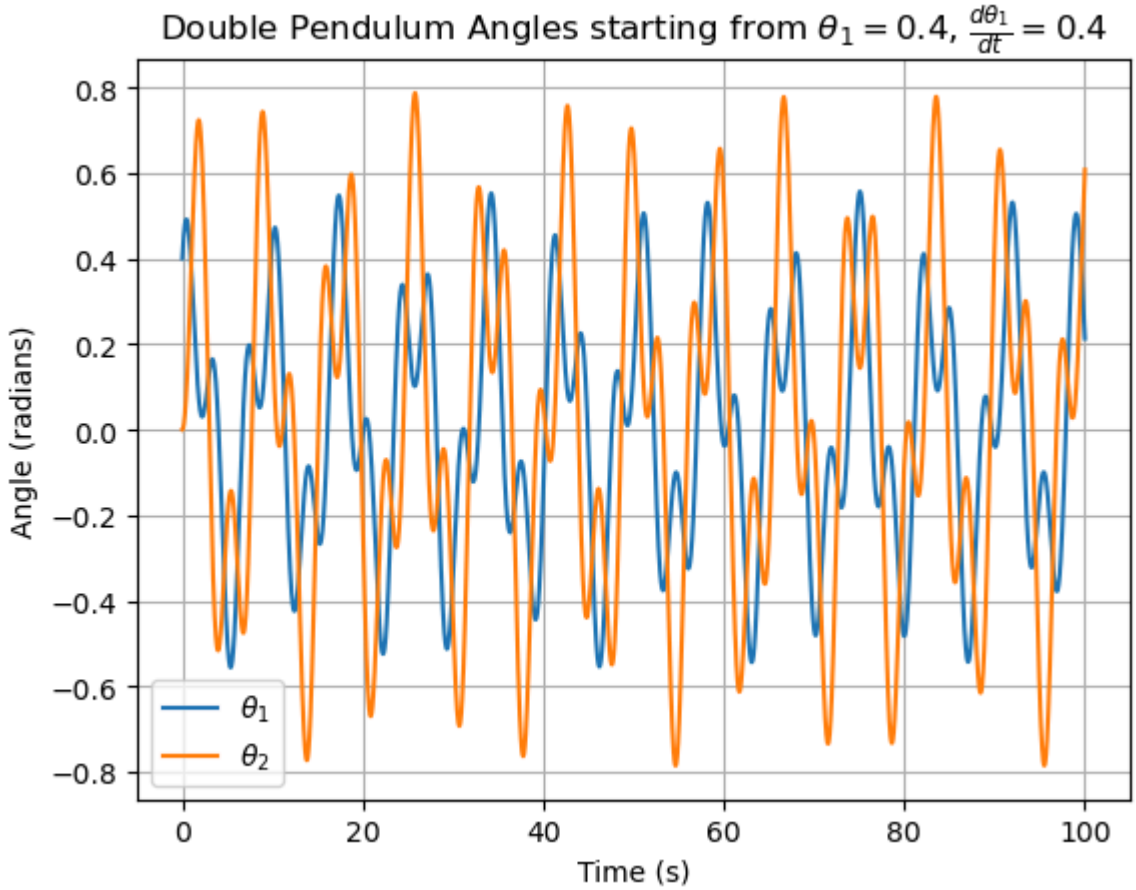


The document was converted with **Code-Format**: https://code-format.com/

In [2]:

```
# Initial condition: small angles and zero velocities
x0 = np.array([0.4, 0, 0.4, 0])

# Time vector
t = np.linspace(0, 100, 5000)

# Solve the ODE
x = odeint(f, x0, t)

# Extract angles
theta1 = x[:, 0]
theta2 = x[:, 1]

# Plot angles
plt.figure()
plt.plot(t, theta1, label=r'$\theta_1$')
plt.plot(t, theta2, label=r'$\theta_2$')
plt.xlabel('Time (s)')
plt.ylabel('Angle (radians)')
plt.legend()
plt.grid(True)
plt.title(r'Double Pendulum Angles starting from $\theta_1 = 0.4$,
$\frac{d \theta_1}{dt} =0.4$')
plt.show()
```

The document was converted with **Code-Format**: https://code-format.com/

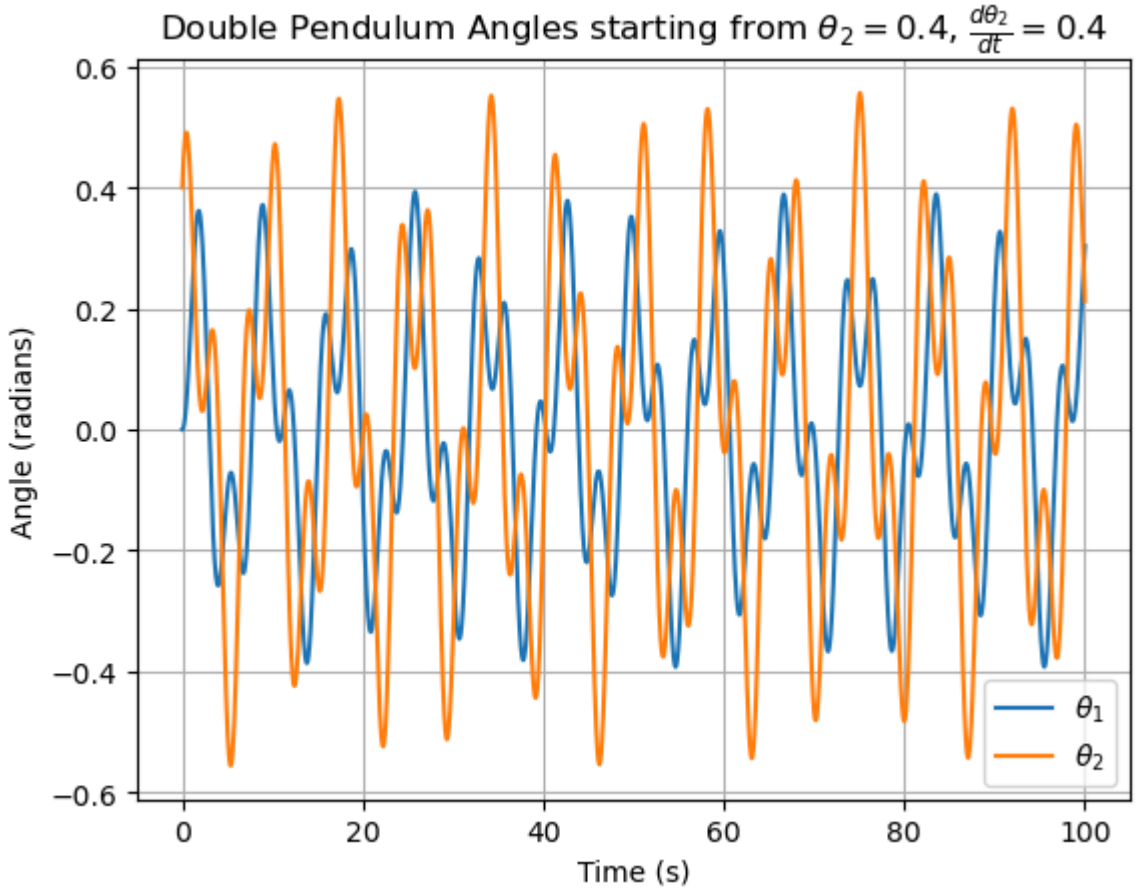


The document was converted with **Code-Format**: https://code-format.com/

In [4]:

```
# Initial condition: small angles and zero velocities
x0 = np.array([0, 0.4, 0, 0.4])

# Time vector
t = np.linspace(0, 100, 5000)

# Solve the ODE
x = odeint(f, x0, t)

# Extract angles
theta1 = x[:, 0]
theta2 = x[:, 1]

# Plot angles
plt.figure()
plt.plot(t, theta1, label=r'$\theta_1$')
plt.plot(t, theta2, label=r'$\theta_2$')
plt.xlabel('Time (s)')
plt.ylabel('Angle (radians)')
plt.legend()
plt.grid(True)
plt.title(r'Double Pendulum Angles starting from $\theta_2 = 0.4$,
$\frac{d \theta_2}{dt} =0.4$')
plt.show()
```

The document was converted with **Code-Format**: https://code-format.com/

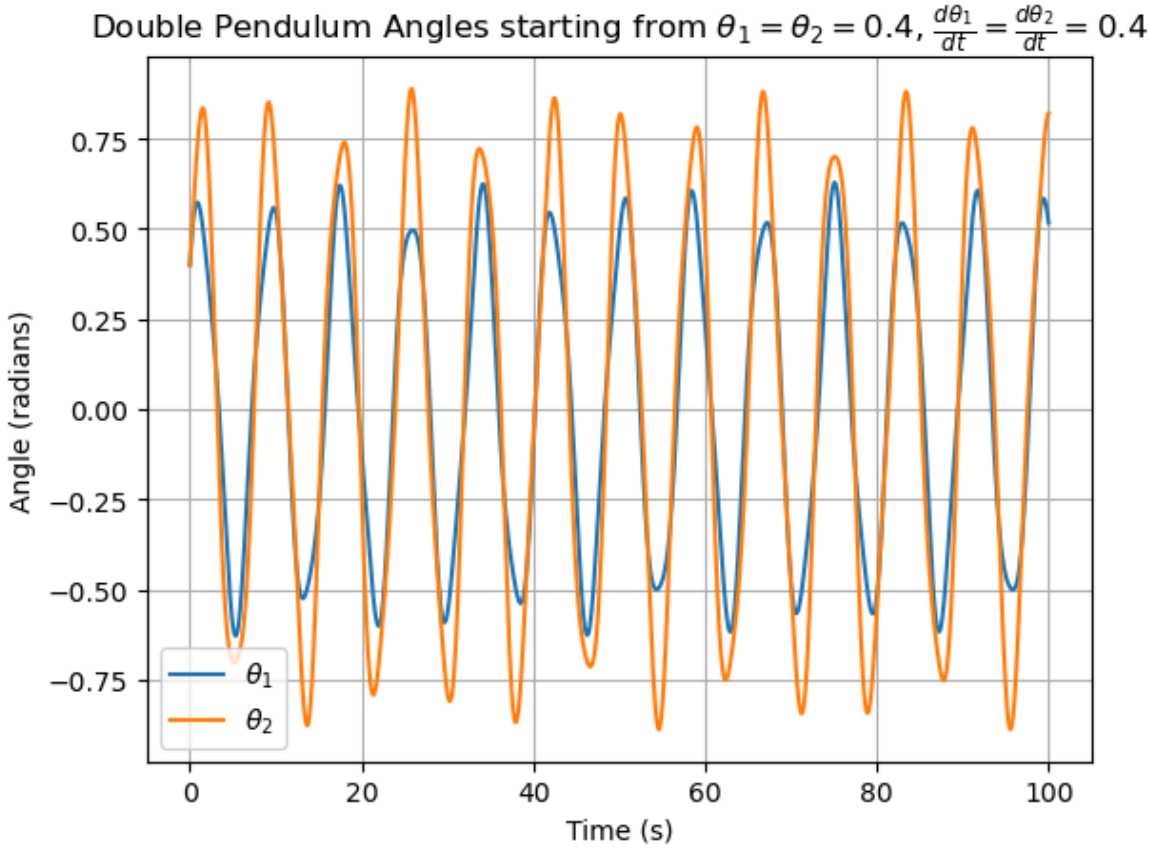


The document was converted with **Code-Format**: https://code-format.com/

In [8]:

```
# Initial condition: small angles and zero velocities
x0 = np.array([0.4, 0.4, 0.4, 0.4])

# Time vector
t = np.linspace(0, 100, 5000)

# Solve the ODE
x = odeint(f, x0, t)

# Extract angles
theta1 = x[:, 0]
theta2 = x[:, 1]

# Plot angles
plt.figure()
plt.plot(t, theta1, label=r'$\theta_1$')
plt.plot(t, theta2, label=r'$\theta_2$')
plt.xlabel('Time (s)')
plt.ylabel('Angle (radians)')
plt.legend()
plt.grid(True)
plt.title(r'Double Pendulum Angles starting from $\theta_1 = \theta_2
= 0.4$, $\frac{d \theta_1}{dt} =\frac{d \theta_2}{dt} =0.4$')
plt.show()
```

The document was converted with **Code-Format**: https://code-format.com/



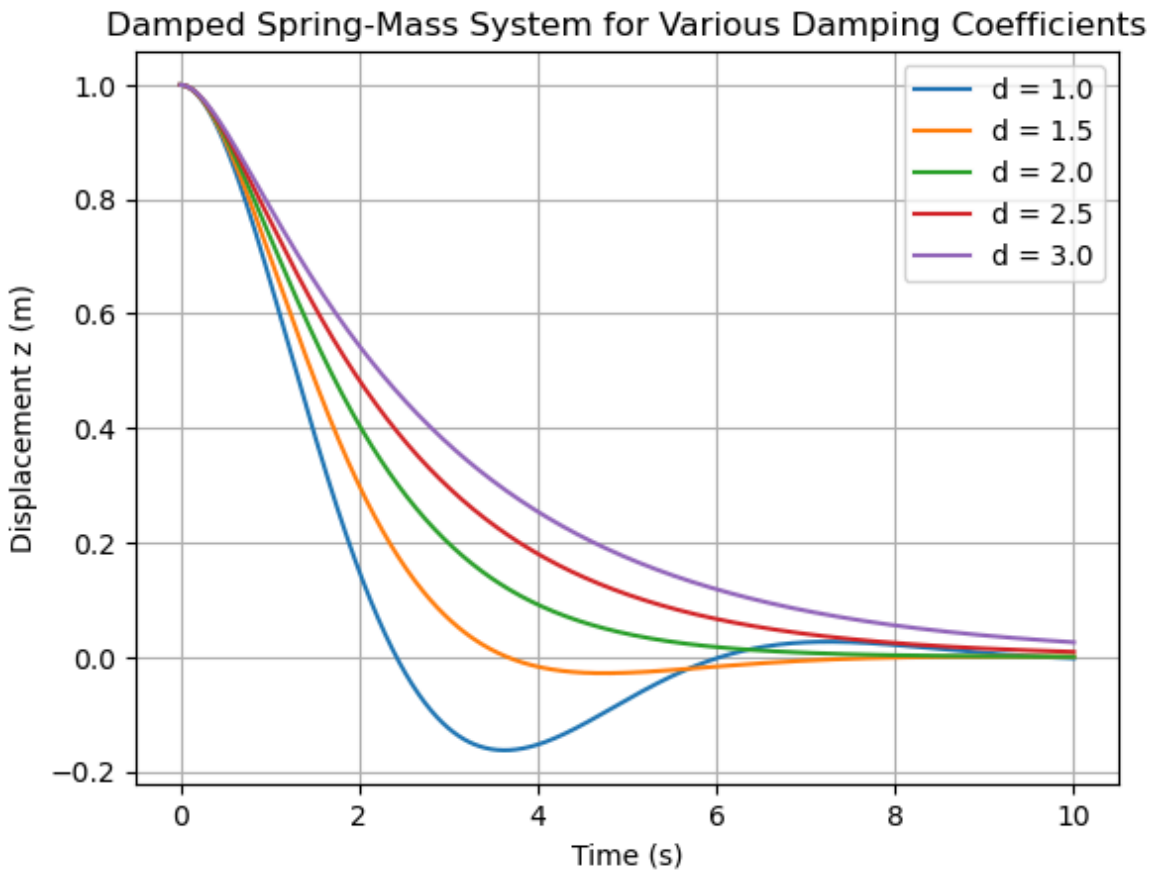
5 - spring mass

The document was converted with **Code-Format**: https://code-format.com/

In [8]:

```
# System parameters
m = 1
k = 1
ds = np.linspace(1, 3, 5)  # damping coefficients to loop over

# Time vector
t = np.linspace(0, 10, 1000)

# Initial condition: z(0) = 1, z_dot(0) = 0
x0 = [1, 0]

plt.figure()
for d in ds:
    # State-space matrix A
    A = np.array([[0, 1],
                  [-k/m, -d/m]])

    # ODE function
    def f(x, t):
        return A @ x

    # Solve ODE
    x = odeint(f, x0, t)

    # x[:,0] = z, x[:,1] = z_dot
    plt.plot(t, x[:,0], label=f'd = {d:.1f}')

plt.xlabel('Time (s)')
plt.ylabel('Displacement z (m)')
plt.title('Damped Spring-Mass System for Various Damping
Coefficients')
plt.legend()
plt.grid(True)
plt.show()
```

The document was converted with **Code-Format**: https://code-format.com/



At $d = 2\sqrt{mk}$, $d = 2$ since $m = k = 1$. So, we are considering the green line. At critical damping, the system returns to equilibrium (displacement $z = 0$) as quickly as possible without any overshooting; as seen by the yellow line ($d = 1.5$), when the system is underdamped (damping less than the critical damping), there is quick return towards equilibrium, but the system overshoots and oscilates around equilibrium before settling. An overdamped system ($d > 2$, red and purple lines) will return to the equilibrium without oscillation but has a slower return to equilibrium than the critically damped system.

To understand why critical damping is ideal, consider a car's shock absorber. If the system is underdamped, the car bounces up and down (oscillates vertically) after hitting a bump. If the system is overdamped, the car slowly returns to its normal/equilibrium position, and may feel "heavy","sticky", or "sluggish." Critical damping provides the optimal balance, where the car quickly stabilizes without bouncing. Critical damping provides the fastest non-oscillatory return to a system's equilibrium.

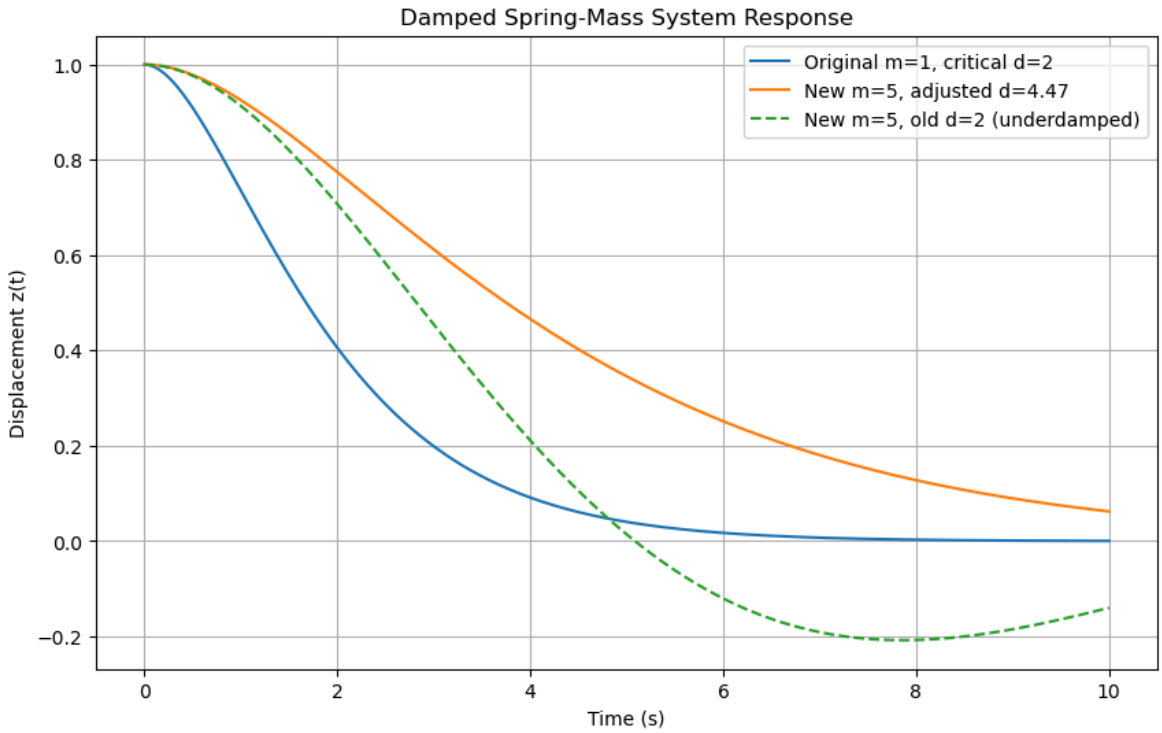
The document was converted with **Code-Format**: https://code-format.com/

In [12]:

```
from scipy.integrate import solve_ivp

# System parameters
m_original = 1
k_original = 1
d_original = 2  # critical damping for m=1, k=1

m_new = 5  # new mass

# Compute new damping for critical damping
d_new = 2 * np.sqrt(m_new * k_original)  # keeping k the same

# State-space representation: x = [z, z_dot], dx/dt = [z_dot, z_ddot]
def damped_spring(t, x, m, d, k):
    z, z_dot = x
    z_ddot = -(d/m)*z_dot - (k/m)*z
    return [z_dot, z_ddot]

# Initial condition
x0 = [1, 0]  # start displaced 1 unit, zero velocity

# Time span
t_span = (0, 10)
t_eval = np.linspace(*t_span, 1000)

# Solve for original system
sol_orig = solve_ivp(damped_spring, t_span, x0, t_eval=t_eval, args=
(m_original, d_original, k_original))

# Solve for new mass with adjusted damping
sol_new = solve_ivp(damped_spring, t_span, x0, t_eval=t_eval, args=
(m_new, d_new, k_original))

# Solve for new mass without adjusting damping (underdamped)
sol_under = solve_ivp(damped_spring, t_span, x0, t_eval=t_eval, args=
(m_new, d_original, k_original))

# Plot results
plt.figure(figsize=(10,6))
plt.plot(sol_orig.t, sol_orig.y[0], label='Original m=1, critical
d=2')
plt.plot(sol_new.t, sol_new.y[0], label=f'New m=5, adjusted d=
{d_new:.2f}')
plt.plot(sol_under.t, sol_under.y[0], '--', label='New m=5, old d=2
(underdamped)')
plt.xlabel('Time (s)')
```

The document was converted with **Code-Format**: https://code-format.com/

```
plt.ylabel('Displacement z(t)')
plt.title('Damped Spring-Mass System Response')
plt.grid(True)
plt.legend()
plt.show()
```



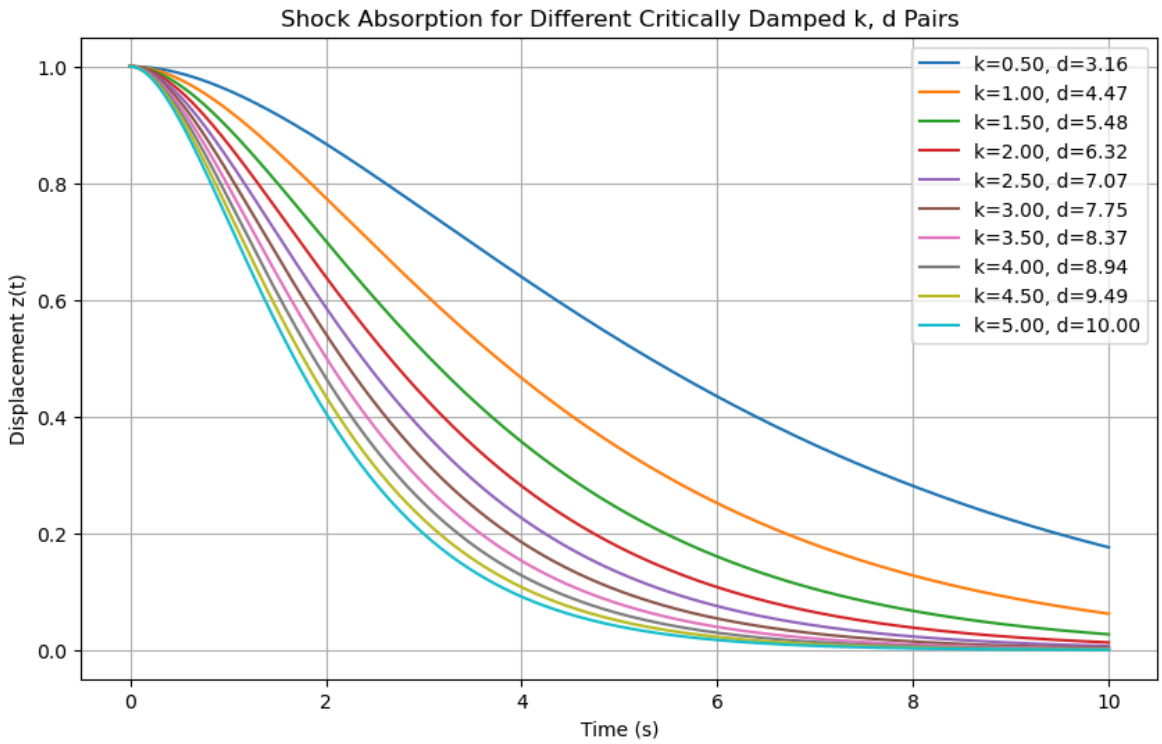
$d_{crit} = 2\sqrt{mk}$. For critical damping, $d_{new} = 2\sqrt{5k_{new}}$.

The document was converted with **Code-Format**: https://code-format.com/

In [16]:

```
# Mass
m_new = 5

# Number of sample pairs
n = 10

# Range of spring constants
k_all = np.linspace(0.5, 5, 100)
d_all = 2 * np.sqrt(m_new * k_all)  # critical damping

# Pick n equally spaced indices
indices = np.linspace(0, len(k_all)-1, n, dtype=int)
k_samples = k_all[indices]
d_samples = d_all[indices]

# Time for simulation
t_span = (0, 10)
t_eval = np.linspace(*t_span, 1000)

# Initial displacement and velocity
z0 = [1, 0]  # z = 1, dz/dt = 0

plt.figure(figsize=(10,6))

for k_i, d_i in zip(k_samples, d_samples):
    # State-space system: x = [z, z_dot]
    def f(t, x):
        z, z_dot = x
        dzdt = z_dot
        dz_dotdt = -(d_i/m_new)*z_dot - (k_i/m_new)*z
        return [dzdt, dz_dotdt]

    sol = solve_ivp(f, t_span, z0, t_eval=t_eval)
    plt.plot(sol.t, sol.y[0], label=f'k={k_i:.2f}, d={d_i:.2f}')

plt.xlabel('Time (s)')
plt.ylabel('Displacement z(t)')
plt.title('Shock Absorption for Different Critically Damped k, d
Pairs')
plt.legend()
plt.grid(True)
plt.show()
```

The document was converted with **Code-Format**: https://code-format.com/



As k increases, d also increases to maintain critical damping. Smaller spring constants k with smaller damping coefficients d lead to slower responses while larger k, d pairs lead to stiffer springs/heavier damping and faster responses. All critically damped systems have no oscilation. The "best" system depends on the application: for comfort, choose a small k and small d, but for fast response, choose large k, d pairs.

The document was converted with **Code-Format**: https://code-format.com/